AN
INTERNSHIP REPORT
on

FRONTEND OPTIMIZATION AND PERFORMANCE ENHANCEMENT

Submitted in partial fulfilment for the award of the Degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

Under the guidance of

Mr. Nalliboyina Ramakrishna

Asst. Professor, Dept of CSE

Submitted By:

ARAVIND GANTEDA (N200183)

Department of Computer Science and Engineering

Rajiv Gandhi University of Knowledge Technologies, Nuzvid

(A.P. Government Act 18 of 2008)

RGUKT-NUZVID, Eluru Dist., Andhra Pradesh – 521202.

2025–2026

RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES NUZVID

(A.P. Government Act 18 of 2008)

RGUKT-NUZVID, Eluru Dist., Andhra Pradesh – 521202.

CERTIFICATE OF COMPLETION

This is to certify that the work entitled, **"Frontend Optimization and Performance Enhancement"** is the bonafide internship work of **Aravind Ganteda (N200183)** carried out under the guidance and supervision of **Mr. Nalliboyina Ramakrishna** for the final year internship of Bachelor of Technology in the Department of Computer Science and Engineering at RGUKT Nuzvid. This internship was carried out at **Gridlex**, under the mentorship of **Mr. Satya Harish (Team Lead)**, during the period **4th August 2025 to 10th April 2026**. This work is completed during the academic year 2025–2026 under our guidance.

Mr. N. Ramakrishna

Project Supervisor
Asst. Professor,
Dept. of CSE,
RGUKT, Nuzvid.

Mr. Udaya Kumar Ambati

Head of the Department
Asst. Professor,
Dept. of CSE,
RGUKT, Nuzvid.

RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES NUZVID

(A.P. Government Act 18 of 2008)

RGUKT-NUZVID, Eluru Dist., Andhra Pradesh – 521202.

CERTIFICATE OF EXAMINATION

This is to certify that the work entitled, **"Frontend Optimization and Performance Enhancement"** is the bonafide internship work of **Aravind Ganteda (N200183)** and hereby accord our approval of it as a study carried out and presented in a manner required for its acceptance in the final year of Bachelor of Technology for which it has been submitted. This approval does not necessarily endorse or accept every statement made, opinion expressed, or conclusion drawn, as recorded in this report. It only signifies the acceptance of this report for the purpose for which it has been submitted.

Mr. N. Ramakrishna

Project Supervisor
Asst. Professor,
Dept. of CSE,
RGUKT, Nuzvid

Examiner 1

Project Examiner
Asst. Professor,
Dept. of CSE,
RGUKT, Nuzvid

Examiner 2

Project Examiner
Asst. Professor,
Dept. of CSE,
RGUKT, Nuzvid

RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES NUZVID

(A.P. Government Act 18 of 2008)

RGUKT-NUZVID, Eluru Dist., Andhra Pradesh – 521202.

DECLARATION

I, **Aravind Ganteda (N200183)**, hereby declare that the internship report entitled **"Frontend Optimization and Performance Enhancement"** done under the guidance of **Mr. N. Ramakrishna** is submitted for the final year of Bachelor of Technology in Computer Science and Engineering at RGUKT – Nuzvid. I also declare that this report is a result of my own effort and has not been copied or imitated from any source. Citations from any websites are mentioned in the references. The results embodied in this internship report have not been submitted to any other university or institute for the award of any degree or diploma.

Date: _____

Place: NUZVID

Aravind Ganteda (N200183)

RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES NUZVID

(A.P. Government Act 18 of 2008)

RGUKT-NUZVID, Eluru Dist., Andhra Pradesh – 521202.

ACKNOWLEDGEMENT

I would like to express my profound gratitude and deep regard to my honourable guide **Mr. N. Ramakrishna** sir for his exemplary guidance, monitoring, and constant encouragement throughout the course of this internship. I am extremely grateful for the confidence bestowed in me and for entrusting me with the internship work entitled **"Frontend Optimization and Performance Enhancement"**. At this juncture, I feel deeply honoured in expressing my sincere thanks to him for making the resources available at the right time and providing valuable insights leading to the successful completion of this internship.

I sincerely thank **Mr. Satya Harish (Team Lead, Gridlex)**, my industry mentor, for his constant support, technical guidance, and invaluable feedback throughout the internship period. His practical insights into frontend engineering and software development greatly shaped the quality and direction of this work.

I am grateful to **Mr. Udaya Kumar Ambati**, Head of the Department of Computer Science and Engineering, RGUKT Nuzvid, and the faculty of our department for their endeavour and extended support throughout my education in this institute.

Last but not least, I thank the Almighty, and I place a deep sense of gratitude to my family members and friends who have been a constant source of inspiration and support during the preparation of this internship report.

Aravind Ganteda (N200183)

Abstract

Modern web applications face a persistent tension between responsiveness and data freshness. As the complexity of single-page applications (SPAs) grows, the number of API calls made on each page load tends to increase commensurately, placing a significant burden on both the network and the backend server. Unoptimized applications suffer from slow Time-to-Interactive (TTI) metrics, excessive spinner states, and poor perceived performance—all of which degrade the user experience and risk user drop-off.

This internship report presents a systematic approach to **Frontend Optimization and Performance Enhancement**, carried out at **Gridlex** during the period August 2025 to April 2026. The work targets four interconnected problem areas: redundant API calls, slow page-load performance, poor reload experience, and a non-scalable frontend architecture. The primary technical contribution is a client-side caching layer built on the browser's **IndexedDB** API, augmented with a **version-based cache invalidation** mechanism that ensures data consistency without requiring manual cache clearing. When the API version token changes, cached entries are automatically detected as stale and refreshed; otherwise, data is served instantly from the local store.

Beyond caching, the work encompasses a comprehensive set of optimizations: API request deduplication and graceful error handling, code refactoring for readability and long-term maintainability, the extraction of a shared reusable component library following atomic design principles, resolution of UI rendering glitches and state inconsistency bugs, and a re-scoped state management strategy that minimizes unnecessary component re-renders. Performance profiling using browser developer tools (Network and Performance tabs) was used throughout to validate improvements empirically.

The resulting system achieves measurably faster initial paint times, near-instant data display on page reload, and a modular architecture that can accommodate growing user load and data volumes.

Keywords: Frontend Optimization, IndexedDB, Client-Side Caching, Cache Invalidation, API Optimization, State Management, Code Refactoring, Performance Profiling, Single-Page Application, Scalability.

Contents

Certificate of Completion **1**

Abstract **5**

1 Introduction **8**
1.1 Background 8
1.2 Problem Statement 9
1.3 Objectives 9
1.4 Organisation of Report 10

2 Literature Survey **12**
2.1 Overview 12
2.2 Browser-Side Storage APIs 12
2.3 Cache Invalidation Strategies 13
2.4 Frontend Performance Optimization Techniques 14
2.5 State Management Patterns 14
2.6 Summary and Motivation 15

3 System Architecture and Methodology **16**
3.1 Overview 16
3.2 Client-Side Caching with IndexedDB 16
3.2.1 Why IndexedDB? 16
3.2.2 Database Initialization 17
3.2.3 Data Flow: Cache-First Strategy 17
3.3 Version-Based Cache Invalidation 17
3.3.1 Motivation 17
3.3.2 Version Token Protocol 18
3.3.3 Benefits 18
3.3.4 Implementation Pseudocode 19
3.4 Performance Optimization 19
3.4.1 Deferred Rendering 19
3.4.2 Request Deduplication 20

3.4.3 Caching for Instant Reload . . . 20
3.5 API Layer Optimization . . . 20
3.5.1 Graceful Error Handling . . . 20
3.5.2 Request Batching and Pagination . . . 20
3.6 Code Refactoring and Component Reusability . . . 21
3.6.1 Refactoring and Readability . . . 21
3.6.2 Reusable Component Library . . . 21
3.6.3 Bug Fixes and Stability Improvements . . . 21
3.7 State Management Optimization . . . 22
3.7.1 Re-render Reduction . . . 22
3.7.2 State Co-location . . . 22
3.8 Scalability Considerations . . . 23
3.9 Performance Profiling Methodology . . . 23

4 Results and Discussion **24**
4.1 Overview . . . 24
4.2 Implementation Details . . . 24
4.2.1 IndexedDB Caching Layer . . . 24
4.2.2 Service Layer Integration . . . 25
4.2.3 State Management Refactoring . . . 25
4.3 Performance Metrics . . . 26
4.3.1 API Call Reduction . . . 26
4.3.2 Page Load and TTI Improvement . . . 26
4.3.3 Re-render Reduction . . . 26
4.4 Challenges Encountered . . . 26
4.4.1 Cache Invalidation Complexity . . . 26
4.4.2 Large Local Data Management . . . 27
4.4.3 API-Cache Synchronization . . . 27
4.5 Trade-offs . . . 27
4.5.1 Performance vs. Data Consistency . . . 27
4.5.2 Storage Growth vs. API Call Frequency . . . 27
4.5.3 Memoization Overhead vs. Re-render Cost . . . 27
4.6 Discussion . . . 28

5 Conclusion and Future Work **29**
5.1 Conclusion . . . 29
5.2 Future Work . . . 30

Chapter 1

Introduction

1.1 Background

The modern web landscape is dominated by single-page applications (SPAs) that must deliver rich, data-driven experiences while maintaining near-native responsiveness. Frameworks such as React, Angular, and Vue have enabled developers to build highly interactive interfaces, but they have simultaneously introduced new performance challenges: component trees that re-render unnecessarily, API layers that issue duplicate requests, and a near-complete absence of local data persistence between sessions.

A typical user navigating a data-heavy SPA will trigger several API calls on every page visit—even when the underlying data has not changed since the last visit. Each of these calls adds latency proportional to network round-trip time, backend processing time, and serialization overhead. In environments with high latency or constrained bandwidth—mobile networks being the canonical example—this pattern translates directly into long blank screens, degraded Time-to-Interactive (TTI), and frustrated users.

The performance problem is compounded at scale. As user traffic grows, backend servers receive proportionally more requests, increasing infrastructure costs and the probability of service degradation under peak load. Solving the problem at the frontend—by intelligently caching data locally and avoiding unnecessary requests—therefore benefits both the end user and the system as a whole.

This project was motivated by the observation that the frontend of a real-world web application was exhibiting exactly these pathologies: redundant network requests on every page load, slow initial rendering, blank screens on reload, and an architecture that was not equipped to handle increasing data volumes or user concurrency. Rather than treating these as isolated bugs, this project addresses them as symptoms of a deeper architectural gap—the absence of a coherent, version-aware client-side caching strategy—and proposes a comprehensive solution.

1.2 Problem Statement

The central challenge addressed by this project is as follows:

> *How can a frontend application be redesigned and optimized to deliver fast, consistent, and data-fresh experiences to users, while simultaneously reducing the burden on backend API servers and ensuring the architecture remains scalable as data volumes and user load grow?*

More specifically, the project addresses four interconnected sub-problems:

1. **Frequent API Calls:** Redundant network requests were being issued on every page load, regardless of whether the underlying data had changed. This caused significant rendering delays and increased server load unnecessarily.
2. **Slow Page Load Performance:** Unoptimized rendering pipelines and heavy synchronous data fetching resulted in poor Time-to-Interactive (TTI) metrics. Users perceived the application as slow and unresponsive, particularly on first load.
3. **Poor Reload Experience:** The application had no mechanism for local data persistence. On every page reload, users faced blank screens or spinner states while API responses were awaited, even for data that had already been fetched in a prior session.
4. **Scalability Concerns:** The frontend architecture had grown organically and was not designed to handle increasing user load or growing data volumes efficiently. State management was scattered, components were tightly coupled, and there was significant code duplication.

1.3 Objectives

The specific objectives of this project are:

O1. Reduce API Calls
Implement an intelligent client-side caching strategy using IndexedDB that minimizes redundant network requests by serving previously fetched data from local storage whenever valid cached data is available.

O2. Improve Load Time
Achieve measurably faster page loads and smoother Time-to-Interactive performance through deferred rendering, cached data injection at startup, and optimized component re-render cycles.

O3. Enhance User Experience
Deliver instant data on page reload and a seamless, responsive interface by eliminating blank screens and spinner delays for data that is available locally.

O4. Build a Scalable Architecture
Design a modular, future-proof frontend capable of handling growing data volumes and user demand through lazy loading, pagination, LRU-inspired cache eviction, and reusable component abstractions.

O5. Improve Code Quality and Maintainability
Refactor existing code to remove duplication, standardize naming conventions, and restructure modules for long-term maintainability; extract shared logic into a reusable component library.

O6. Stabilize State Management
Reduce unnecessary component re-renders by refining state update logic, scoping state closer to where it is consumed, and applying memoization where appropriate.

O7. Validate Improvements Empirically
Use browser developer tools—including the Performance and Network tabs—to profile the application before and after each optimization, ensuring that improvements are data-driven rather than speculative.

1.4 Organisation of Report

This report is organized into five chapters.

Chapter 1 – Introduction (this chapter) provides the background and motivation, defines the problem statement, states the specific objectives, and outlines the report structure.

Chapter 2 – Literature Survey reviews the foundational concepts and prior work underpinning the project, covering browser storage APIs, caching strategies, cache invalidation patterns, and performance optimization techniques.

Chapter 3 – System Architecture and Methodology presents the complete technical design of the optimized frontend system, including the caching architecture, version-based cache invalidation mechanism, API optimization strategy, state management approach, and code quality improvements.

Chapter 4 – Results and Discussion presents the implementation details, performance measurements, and a critical analysis of outcomes, including challenges

encountered and trade-offs made.

Chapter 5 – Conclusion and Future Work summarizes the key contributions and outlines a concrete roadmap for future enhancements.

Chapter 2

Literature Survey

2.1 Overview

The challenge of optimizing frontend web application performance has produced a rich body of research and industry best practices spanning caching theory, browser storage APIs, state management patterns, and progressive enhancement strategies. This chapter reviews the foundational literature and prior work that informed the design of the system presented in this project. The discussion is organized into four thematic areas: (1) browser-side storage and caching APIs, (2) cache invalidation strategies, (3) frontend performance optimization techniques, and (4) state management patterns.

2.2 Browser-Side Storage APIs

Modern browsers expose several mechanisms for client-side data persistence. **Cookies**, the earliest form of browser storage, are limited to approximately 4 KB per domain and are transmitted with every HTTP request, making them unsuitable for caching large API payloads. **Web Storage** (`localStorage` and `sessionStorage`) provides a synchronous key-value store with a per-origin limit of approximately 5–10 MB. While convenient for small amounts of data, the synchronous nature of the Web Storage API blocks the main thread during read and write operations, creating potential jank in high-frequency update scenarios.

IndexedDB is a low-level, transactional, key-value database built into the browser. Unlike Web Storage, IndexedDB is *asynchronous*—all read and write operations are performed off the main thread via a callback or Promise-based API, ensuring that storage operations never block UI rendering. IndexedDB supports structured data (including `File`, `Blob`, and `ArrayBuffer` objects), indexed queries, and storage quotas that typically range from 50 MB to several gigabytes depending on available disk space and browser policy. These characteristics make IndexedDB the most appropriate

browser storage technology for caching large, structured API response payloads.

The Cache Storage API, a newer addition to the browser's storage ecosystem, is closely associated with Service Workers and is designed primarily for caching HTTP responses (network resources such as scripts, stylesheets, and images) rather than structured application data. For the specific use case of caching JSON API responses in a structured, queryable format, IndexedDB remains the most appropriate choice.

2.3 Cache Invalidation Strategies

Cache invalidation—the process of determining when cached data is stale and should be refreshed from the authoritative source—is widely regarded as one of the hardest problems in computer science. Several strategies have been developed to address it:

Time-to-Live (TTL) Expiry assigns each cached entry a timestamp and a maximum age. When a cached entry is retrieved, its age is compared against the TTL threshold; if it has expired, the entry is considered stale and a fresh API request is issued. TTL-based expiry is simple to implement but can result in either stale data (if the TTL is too long) or excessive API calls (if the TTL is too short).

ETag and Last-Modified Headers are HTTP cache control mechanisms that allow the server to signal whether its response has changed since the client last fetched it. The client sends a conditional request (using `If-None-Match` or `If-Modified-Since` headers), and the server responds either with a `304 Not Modified` status (no data transfer required) or a full `200 OK` response with the updated payload. This approach minimizes unnecessary data transfer but still requires a network round-trip for every cache validation check.

Version-Based Cache Invalidation embeds an explicit version token in the API response (or in a lightweight version-check endpoint). The client stores the version token alongside the cached data. On each application load, the client either reads a locally persisted version or fetches the current version from a lightweight endpoint, compares it with the stored version, and invalidates the cache only when the versions differ. This approach eliminates round-trips for data that has not changed, while guaranteeing that stale data is never served after a server-side update.

Least Recently Used (LRU) Eviction is a cache replacement policy that discards the least recently accessed entries when the cache reaches its size limit. LRU eviction prevents storage bloat while retaining the entries most likely to be requested again, making it particularly effective for applications with skewed access patterns.

2.4 Frontend Performance Optimization Techniques

Performance optimization of web frontends encompasses a broad range of techniques. Key techniques relevant to this project include:

Deferred and Lazy Rendering: Deferring the rendering of below-the-fold content and lazy-loading data only when it enters the viewport reduces the initial render workload and improves First Contentful Paint (FCP) and TTI metrics.

Request Deduplication: In applications where multiple components independently trigger the same API request, request deduplication logic ensures that only one in-flight request exists for a given resource at any time. Subsequent components subscribe to the result of the existing request rather than initiating a new one.

Memoization and Selective Re-rendering: React's `useMemo` and `useCallback` hooks, as well as the `React.memo` higher-order component, allow developers to cache computed values and prevent child components from re-rendering when their props have not changed. These techniques are most effective when applied judiciously—over-memoization can itself become a source of overhead.

Code Splitting and Tree Shaking: Bundlers such as Webpack and Vite can split application code into smaller chunks that are loaded on demand, and can eliminate unused code through static analysis (tree shaking). Both techniques reduce the initial JavaScript bundle size, accelerating parse and execution time.

2.5 State Management Patterns

State management in large React applications has evolved considerably. Global state management libraries such as Redux, MobX, and Zustand have been widely adopted, but they introduce the risk of over-centralization—placing all application state in a single global store and causing components to re-render on every state change, even when the change is irrelevant to the component in question.

The contemporary consensus favors *co-located state*: state should live as close as possible to the component that consumes it, escalating to a higher scope only when genuinely shared. Context API is appropriate for low-frequency global state (such as authentication and theme); local component state (`useState`, `useReducer`) is preferred for high-frequency, component-specific state. This approach, combined with memoization, minimizes the number of components that re-render on any given state update.

2.6 Summary and Motivation

The literature establishes that version-based cache invalidation, combined with IndexedDB as the persistence layer, represents a robust and practical approach to client-side caching of API responses. Co-located state management and targeted memoization address the re-render problem. Request deduplication and graceful error handling improve API layer reliability. These insights collectively motivate the system design presented in Chapter 3.

Chapter 3

System Architecture and Methodology

3.1 Overview

This chapter presents the complete technical architecture of the optimized frontend system. The system is designed around a layered data-access pattern: the UI layer communicates exclusively with a service layer, which in turn consults the IndexedDB cache before issuing any network request. The cache is governed by a version-based invalidation protocol that guarantees data freshness without unnecessary round-trips. Surrounding this core caching infrastructure, a set of complementary optimizations addresses API reliability, code quality, state management, and system scalability.

The architecture can be conceptualized as a pipeline:

UI Component $\longrightarrow$ **Service Layer** $\longrightarrow$ **Cache Decision** $\longrightarrow$ **IndexedDB / API**

Each stage of this pipeline is described in detail in the following sections.

3.2 Client-Side Caching with IndexedDB

3.2.1 Why IndexedDB?

IndexedDB was selected as the caching storage backend for three primary reasons. First, it is *asynchronous*: all read and write operations are non-blocking and do not interfere with the main thread or the UI rendering pipeline. Second, it supports *structured data*: entire JSON API response payloads—including nested objects and arrays—can be stored and retrieved without serialization overhead. Third, it provides *large storage quotas*: unlike `localStorage` (capped at approximately 5 MB), IndexedDB can store tens to hundreds of megabytes of data, making it suitable for caching substantial API responses.

3.2.2 Database Initialization

The IndexedDB database is initialized when the application first loads. A single object store (analogous to a table in a relational database) is created to hold cached API responses. Each record in the object store contains:

- **key**: A string identifier derived from the API endpoint and query parameters.
- **data**: The full JSON payload returned by the API.
- **version**: The version token associated with the data at the time of caching.
- **timestamp**: The UTC timestamp of the last successful cache write, used for TTL-based expiry as a secondary safeguard.

3.2.3 Data Flow: Cache-First Strategy

The data access pattern follows a cache-first strategy:

1. The UI component requests data by calling a service function (e.g., `fetchDashboardData()`).
2. The service layer queries IndexedDB for a cached record matching the request key.
3. If a valid cached record is found (and its version is current—see Section 3.3), the cached data is returned immediately to the UI component without issuing a network request.
4. If no valid cached record exists (cache miss or version mismatch), the service layer issues the API request, receives the response, stores it in IndexedDB, and returns it to the UI component.

This strategy ensures that the UI receives data as quickly as possible—from the local cache on all but the first request or after a version change—while maintaining consistency with the server-side data through the version check.

3.3 Version-Based Cache Invalidation

3.3.1 Motivation

A pure TTL-based invalidation scheme has a fundamental limitation: it cannot distinguish between a cache entry that is stale because the server-side data has changed and one that is stale merely because a fixed time interval has elapsed. If the TTL is long, users may see outdated data after a server-side update. If the TTL is short, the cache provides little benefit because fresh API requests are issued frequently.

Version-based cache invalidation addresses this limitation by making the staleness decision based on a semantically meaningful signal—the API data version—rather than an arbitrary time threshold.

3.3.2 Version Token Protocol

The version-based invalidation mechanism operates as follows:

1. Each API response (or a lightweight version-check endpoint) includes a version token—a monotonically increasing integer or a content hash that changes whenever the underlying data is updated on the server.
2. When the frontend stores an API response in IndexedDB, it stores the associated version token alongside the data.
3. On each subsequent application load (or on each data access, depending on the configured strategy), the client fetches the current version token from the version-check endpoint (a lightweight request returning only a version string, not the full payload).
4. The client compares the fetched version token against the version token stored in IndexedDB.
5. If the versions match, the cached data is considered fresh and is served to the UI without issuing the full API request.
6. If the versions differ, the cached entry is invalidated, a full API request is issued to fetch the updated data, and the new data and version token are written to IndexedDB, replacing the stale record.

3.3.3 Benefits

This protocol delivers three important guarantees:

- **Correctness**: Stale data is never served to the user after a server-side update, regardless of how recently the data was cached.
- **Efficiency**: Full API requests are only issued when the data has actually changed. If the server data has not changed, all page reloads are served from the local cache with only the cost of a lightweight version check.
- **Transparency**: Cache invalidation is fully automatic. Users do not need to manually clear caches, and developers do not need to embed cache-busting logic at the call site.

3.3.4 Implementation Pseudocode

The following pseudocode illustrates the core cache decision logic:

```
async function fetchWithCache(key, apiUrl) {
  // Step 1: Load cached record from IndexedDB
  const cached = await idb.get(key);

  // Step 2: Fetch current version token (lightweight request)
  const { version: serverVersion } = await api.get('/version');

  // Step 3: Cache hit and version match
  if (cached && cached.version === serverVersion) {
    return cached.data;   // Serve from cache     no API call
  }

  // Step 4: Cache miss or version mismatch     fetch full data
  const freshData = await api.get(apiUrl);

  // Step 5: Update IndexedDB with fresh data and new version
  await idb.put(key, {
    data: freshData,
    version: serverVersion,
    timestamp: Date.now()
  });

  return freshData;
}
```

Listing 3.1: Version-Based Cache Invalidation Logic

3.4 Performance Optimization

3.4.1 Deferred Rendering

Heavy components that are not required for the initial viewport are rendered lazily using code splitting at the route level. This reduces the JavaScript parse and execution time required before the application becomes interactive, improving the First Contentful Paint (FCP) and Time-to-Interactive (TTI) metrics.

3.4.2 Request Deduplication

In the original system, multiple components on the same page would independently trigger the same API request during their mount lifecycle, resulting in several in-flight requests for identical data. A request deduplication layer was introduced in the service layer: when a request for a given endpoint is already in flight, subsequent requests for the same endpoint subscribe to the result of the existing request rather than initiating a new one. This is implemented using a shared in-memory map of pending request Promises.

3.4.3 Caching for Instant Reload

By persisting API responses in IndexedDB between sessions, the application can inject cached data into the component tree immediately on page load, before the version check has completed. This eliminates blank screen states and provides users with a meaningful, interactive view of their data within milliseconds of navigation—even on slow networks. If the subsequent version check reveals that the cache is stale, the UI is updated in the background with the fresh data.

3.5 API Layer Optimization

3.5.1 Graceful Error Handling

The original application lacked a consistent error handling strategy. API failures resulted in unhandled promise rejections and broken UI states. A centralized error handling middleware was introduced that:

- Catches network errors and server-side error responses (4xx, 5xx status codes).
- Implements an exponential backoff retry mechanism for transient failures (e.g., `503 Service Unavailable`).
- Falls back to the most recently cached data when the network is unavailable, ensuring the application remains functional in degraded network conditions.
- Displays user-friendly error messages in place of cryptic stack traces.

3.5.2 Request Batching and Pagination

For endpoints that return large datasets, pagination was introduced to limit the volume of data transferred in a single request. The frontend requests data in pages of a configurable size and caches each page independently in IndexedDB, keyed by

endpoint and page number. This prevents memory overload in both the browser and the cache store, and allows the UI to render the first page of results while subsequent pages are fetched in the background.

3.6 Code Refactoring and Component Reusability

3.6.1 Refactoring and Readability

A systematic code audit identified the following quality issues in the existing codebase:

- Duplicated data-fetching logic scattered across multiple components.
- Inconsistent naming conventions for variables, functions, and component files.
- Monolithic components that combined data fetching, business logic, and rendering in a single file, making them difficult to test or reuse.
- Unused imports and dead code paths that increased bundle size.

Refactoring addressed each of these issues: data-fetching logic was extracted into custom hooks (`useFetchData`), naming conventions were standardized across the codebase, large components were decomposed into smaller, single-responsibility units, and dead code was removed.

3.6.2 Reusable Component Library

Following atomic design principles, a shared component library was developed comprising:

- **Atoms**: Basic building blocks such as buttons, input fields, badges, and spinner indicators.
- **Molecules**: Composite components built from atoms, such as form groups, data table rows, and notification banners.
- **Organisms**: Complex, self-contained UI sections such as data tables with pagination and filter panels.

By consuming components from this shared library, page-level components avoid duplicating UI logic and benefit automatically from any improvements or bug fixes applied to the shared components.

3.6.3 Bug Fixes and Stability Improvements

The refactoring process surfaced and resolved a number of stability issues:

- **Null and Undefined Data Edge Cases**: Components that accessed properties on API response objects without null checks would throw runtime errors when optional fields were absent. Defensive access patterns and optional chaining (`?.`) were applied throughout.
- **UI Rendering Glitches**: Race conditions between asynchronous data fetches and component unmounts caused spurious state updates on unmounted components. Cleanup functions were added to `useEffect` hooks to cancel or ignore in-flight requests after unmount.
- **State Inconsistency Bugs**: In several cases, UI state derived from API data became out of sync with the actual data when updates were applied. Derived state was replaced with computed values calculated directly from the source of truth on each render.

3.7 State Management Optimization

3.7.1 Re-render Reduction

Unnecessary component re-renders were identified as a significant source of UI jank. The primary causes were:

1. **Overly broad state updates**: A single global state object was being replaced on each API response, triggering re-renders in all components subscribed to any part of that state.
2. **Unstable prop references**: Inline object literals and anonymous functions passed as props were creating new references on every parent render, causing child components to re-render even when their logical inputs had not changed.

These issues were resolved by splitting the global state into smaller, domain-specific slices; applying `React.memo` to pure child components; stabilizing callback references with `useCallback`; and caching expensive computed values with `useMemo`.

3.7.2 State Co-location

State that was previously stored in a global context but consumed by only a single component was moved to local component state. This *state co-location* reduces the number of subscribers notified on each state update, directly reducing re-render frequency.

3.8 Scalability Considerations

The architecture was designed with future growth in mind. Several mechanisms ensure that performance remains acceptable as data volumes and user concurrency increase:

- **Paginated Data Patterns**: Lazy loading and pagination prevent memory overload regardless of the total dataset size.
- **TTL-Based Expiry as a Backstop**: In addition to version-based invalidation, a maximum TTL is applied to all cached entries. This prevents IndexedDB from accumulating large volumes of data that are never actively invalidated (e.g., for API endpoints where the version token never changes).
- **LRU-Inspired Cache Eviction**: When the total size of IndexedDB content exceeds a configurable threshold, a cleanup routine evicts the least recently accessed entries, ensuring that the cache store does not grow without bound.
- **Modular Architecture**: The refactored codebase separates concerns clearly (UI, service logic, caching), making it straightforward to replace or upgrade individual layers—for example, replacing the IndexedDB layer with a Service Worker cache—without disrupting the rest of the application.

3.9 Performance Profiling Methodology

All optimizations were validated empirically using browser developer tools. The **Network tab** was used to measure the number of API requests issued per page load, total data transferred, and request latency before and after each change. The **Performance tab** was used to capture flame graphs of the render pipeline, identifying long tasks, slow render cycles, and layout thrashing. Chrome's **Lighthouse** audit tool provided standardized scores for Performance, Accessibility, and Best Practices, offering a reproducible benchmark for comparing the application state before and after optimization.

Chapter 4

Results and Discussion

4.1 Overview

This chapter presents the implementation details, performance measurements, and a critical analysis of the outcomes achieved by the optimization efforts described in Chapter 3. Results are organized around the four problem areas identified in the Problem Statement: API call frequency, page load performance, reload experience, and code quality. The chapter concludes with an honest discussion of the challenges encountered and the trade-offs inherent in the chosen approach.

4.2 Implementation Details

4.2.1 IndexedDB Caching Layer

The IndexedDB caching layer was implemented as a standalone utility module (`idbCache.js`) that exposes a clean Promise-based API:

```
const DB_NAME = 'appCache';
const STORE   = 'apiResponses';
const VERSION = 1;

function openDB() {
  return new Promise((resolve, reject) => {
    const req = indexedDB.open(DB_NAME, VERSION);
    req.onupgradeneeded = e => {
      e.target.result.createObjectStore(STORE);
    };
    req.onsuccess = e => resolve(e.target.result);
    req.onerror   = e => reject(e.target.error);
  });
}
```

```

export async function cacheGet(key) {
  const db = await openDB();
  return new Promise((resolve, reject) => {
    const tx  = db.transaction(STORE, 'readonly');
    const req = tx.objectStore(STORE).get(key);
    req.onsuccess = e => resolve(e.target.result ?? null);
    req.onerror   = e => reject(e.target.error);
  });
}

export async function cachePut(key, value) {
  const db = await openDB();
  return new Promise((resolve, reject) => {
    const tx  = db.transaction(STORE, 'readwrite');
    const req = tx.objectStore(STORE).put(value, key);
    req.onsuccess = () => resolve();
    req.onerror   = e => reject(e.target.error);
  });
}
```

Listing 4.1: IndexedDB Cache Utility Module

4.2.2 Service Layer Integration

The `fetchWithCache` orchestrator function integrates the IndexedDB layer with the API client, implementing the version-based invalidation protocol described in Section 3.3. API-calling components were refactored to invoke this function rather than calling the API client directly, ensuring that all data access is routed through the caching layer uniformly.

4.2.3 State Management Refactoring

Global state was decomposed into three domain-specific contexts: `AuthContext` (authentication state), `DataContext` (fetched entity data), and `UIContext` (transient UI state such as modal visibility and active filters). Components subscribe only to the context they require, limiting re-render propagation to components genuinely affected by a given state change.

4.3 Performance Metrics

4.3.1 API Call Reduction

The most significant measurable outcome of the caching layer is the reduction in API calls per page load. Before optimization, every page load issued the full suite of API requests regardless of prior visits. After optimization, all requests are served from the IndexedDB cache on repeat visits where the server-side version has not changed. This represents a near-total elimination of redundant network requests on page reload under stable data conditions.

4.3.2 Page Load and TTI Improvement

By serving data from IndexedDB on reload, the application eliminates the waiting time for API responses during the initial render. The UI is populated with cached data immediately on component mount, before any network activity completes. This transforms the user experience from a blank/spinner state to a content-populated state at near-zero latency on repeat visits.

4.3.3 Re-render Reduction

The state management refactoring, combined with the application of `React.memo` and `useCallback`, produced a measurable reduction in the number of component re-renders per user interaction. Profiling using the React DevTools Profiler confirmed that components that previously re-rendered on every global state update now re-render only when their direct dependencies change.

4.4 Challenges Encountered

4.4.1 Cache Invalidation Complexity

Implementing reliable cache invalidation proved more complex than anticipated. The primary challenge was ensuring that stale data is detected and replaced without disrupting active user sessions. Concurrency edge cases—such as multiple browser tabs running the application simultaneously, each potentially holding a different version of the cached data—required careful handling. The solution adopted was to perform the version check on each tab load and to treat each tab's cache independently, accepting the possibility of brief periods of inconsistency between tabs as an acceptable trade-off for simplicity.

4.4.2 Large Local Data Management

As the number of cached entries grew, IndexedDB storage consumption became a concern. Without an eviction strategy, the cache would grow indefinitely. The LRU-inspired cleanup routine described in Section 3.7 addresses this, but correctly implementing LRU order tracking within IndexedDB (which does not natively maintain access-order metadata) required storing a `lastAccessed` timestamp on each record and sorting by it during cleanup.

4.4.3 API-Cache Synchronization

Keeping the local cache synchronized with rapidly changing server-side data is inherently difficult. The version-based invalidation protocol works well when changes are versioned at the endpoint level, but some endpoints in the existing API did not expose version tokens. For these endpoints, a conservative TTL-based fallback was applied, accepting the trade-off of occasional brief data staleness in exchange for avoiding unnecessary API calls.

4.5 Trade-offs

4.5.1 Performance vs. Data Consistency

The cache-first strategy prioritizes performance (serving data instantly from the local cache) over strict data consistency (always serving the most current server-side data). This trade-off is acceptable for the use case in question—dashboard data and reference data that changes infrequently—but would not be appropriate for real-time or safety-critical data where even brief staleness is unacceptable.

4.5.2 Storage Growth vs. API Call Frequency

Caching more data reduces API call frequency but increases IndexedDB storage consumption. The TTL and LRU eviction policies provide the lever for tuning this trade-off; shorter TTLs and more aggressive eviction reduce storage consumption at the cost of more frequent API calls. The appropriate balance must be determined empirically for each deployment context.

4.5.3 Memoization Overhead vs. Re-render Cost

Memoization with `React.memo` and `useMemo` is not free: it introduces a shallow comparison cost on every render and retains objects in memory for longer. For

components with simple, inexpensive render functions, memoization can introduce more overhead than it saves. The strategy adopted was to apply memoization selectively, guided by profiler data, rather than blanket-applying it to every component.

4.6 Discussion

The results confirm that a carefully designed, version-aware client-side caching layer can substantially improve the perceived performance of a data-heavy SPA without requiring any changes to the backend API. The combination of IndexedDB persistence, version-based invalidation, and request deduplication addresses the root causes of the identified performance problems rather than merely masking their symptoms.

The code quality improvements—refactoring, component library extraction, bug resolution—provide a foundation for sustainable long-term development. A modular, well-documented codebase reduces the cost of future changes and makes it easier to onboard new developers without inadvertently reintroducing performance regressions.

The challenges encountered, particularly around cache invalidation correctness and IndexedDB storage management, highlight that client-side caching is not a trivially solved problem. It requires ongoing attention to ensure that the cache remains a net benefit rather than a source of subtle consistency bugs.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

This project designed, implemented, and validated a comprehensive set of frontend optimization techniques for a real-world single-page application. The system addresses the four interconnected problem areas identified in the problem statement—frequent API calls, slow page load performance, poor reload experience, and scalability concerns—through a cohesive set of architectural improvements.

The four key contributions of this project are:

1. **An IndexedDB-based client-side caching layer** that persists structured API response payloads locally in the browser, enabling near-instant data delivery on page reload and eliminating redundant network requests for unchanged data.
2. **A version-based cache invalidation protocol** that guarantees data freshness automatically, without TTL-based guesswork or manual cache clearing, by comparing a lightweight server-side version token against the version stored in the local cache.
3. **A suite of API and rendering optimizations** including request deduplication, graceful error handling with exponential backoff retry, deferred rendering, and pagination, collectively improving Time-to-Interactive and network efficiency.
4. **A refactored, maintainable codebase** with a shared atomic component library, standardized naming conventions, co-located state management, and targeted memoization—reducing unnecessary re-renders and providing a solid foundation for future development.

The results demonstrate that a well-designed client-side caching strategy, even without any backend modifications, can deliver substantial performance improvements and a significantly improved user experience. The modular architecture ensures that these

improvements remain maintainable and extensible as the application grows.

5.2 Future Work

F1. Service Worker Integration

Introducing a Service Worker would enable offline-first capability by intercepting network requests at the browser level and serving cached responses even when the device is completely offline. The Service Worker cache could complement the IndexedDB cache, handling static assets (scripts, stylesheets, images) while IndexedDB continues to handle dynamic API responses.

F2. Background Synchronization

The Background Sync API, available in browsers supporting Service Workers, would allow the application to queue write operations (form submissions, data updates) that fail due to network unavailability and replay them automatically when connectivity is restored. This would make the application genuinely resilient to intermittent network conditions.

F3. Intelligent Request Queuing

A request queue that batches concurrent requests for the same resource and deduplicates them at the network boundary would further reduce redundant API calls beyond what the current in-memory deduplication achieves. The queue could also implement priority ordering, ensuring that user-initiated requests are resolved before background prefetch requests.

F4. Automated Cache Performance Monitoring

Integrating a lightweight telemetry layer that records cache hit/miss rates, eviction frequency, and storage consumption would provide ongoing visibility into cache effectiveness and alert developers to regressions or misconfigured TTL parameters.

F5. Server-Sent Events for Real-Time Invalidation

For use cases where data changes frequently and low latency is important, Server-Sent Events (SSE) could be used to push invalidation signals from the server to the client in real time, triggering targeted cache updates for specific keys without requiring the client to poll the version-check endpoint.

F6. Extended Component Library and Design System

The shared component library developed in this project could be evolved into a full design system with documented usage guidelines, accessibility audits, and a Storybook-based component explorer, reducing onboarding friction for new developers and ensuring visual consistency across the application.

F7. End-to-End Performance Testing

Integrating automated performance testing into the CI/CD pipeline—using tools such as Lighthouse CI or WebPageTest—would prevent future performance regressions from reaching production by enforcing minimum performance budgets on every pull request.

Bibliography

[1] W3C, "Indexed Database API 3.0 Specification," World Wide Web Consortium, 2023. [Online]. Available: https://www.w3.org/TR/IndexedDB-3/

[2] MDN Web Docs, "IndexedDB API," Mozilla Developer Network. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API

[3] MDN Web Docs, "Service Worker API," Mozilla Developer Network. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API

[4] Google Developers, "Performance," web.dev. [Online]. Available: https://web.dev/performance/

[5] React Team, "React.memo," React Documentation. [Online]. Available: https://react.dev/reference/react/memo

[6] R. L. Mattson, J. S. Gecsei, D. R. Slutz, and I. L. Traiger, "Evaluation techniques for storage hierarchies," *IBM Systems Journal*, vol. 9, no. 2, pp. 78–117, 1970.

[7] P. V. Mockapetris, "Domain names – implementation and specification," IETF RFC 1035, 1987.

[8] B. Frost, *Atomic Design*. Pittsburgh, PA: Brad Frost, 2016. [Online]. Available: https://atomicdesign.bradfrost.com/

[9] Google Developers, "Resilient through offline — service workers and the Background Sync API," web.dev, 2021. [Online]. Available: https://web.dev/background-sync/

[10] Google Developers, "Lighthouse," Google Chrome for Developers. [Online]. Available: https://developer.chrome.com/docs/lighthouse/

[11] R. Fielding *et al.*, "Hypertext Transfer Protocol (HTTP/1.1): Caching," IETF RFC 7234, 2014.

[12] D. Abramov and A. Clark, "Redux: A Predictable State Container for JavaScript Apps," 2015. [Online]. Available: https://redux.js.org/